

Tema 2 - Elementos sintácticos del lenguaje

Laboratorio Guiado

Objetivos	Practicar la sintaxis básica de Python con scripts y notebooks: comentarios, variables, tipos de datos, conversiones, cadenas, operadores, condicionales y bucles.
Material necesario	Terminal, Python 3.13, venv del Tema02, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en <code>~/Curso_Python/Tema02</code> .

Laboratorio 1. Comentarios y docstrings

1. Actualizar el repositorio y abrir la carpeta del tema y activar el entorno virtual (El venv ya está creado).

```
git pull origin main
cd ~/Curso_Python/Tema02
source .venv/bin/activate
python --version
code .
```

2. Abrir el fichero `01_comentarios.py` y revisar el uso de docstrings con comillas triples.

Los docstrings permiten escribir bloques de texto multilínea. En este ejercicio se usan como cabecera explicativa del script, aunque más adelante también se usarán en funciones, clases y módulos.

```
'''
    Los comentarios en bloque o Docstring permiten un bloque
    de texto largo delimitando 3 comillas simples o dobles
    al principio y cerrando con las mismas comillas al final
'''

""" Esto es otro ejemplo de Docstring
    pero en este caso con comillas dobles
"""
```

3. Revisar los comentarios de línea con almohadilla y comprobar qué partes se ejecutan.

```
# Cualquier cadena a la derecha de un carácter almohadilla
# se considera un comentario
print('Este print se ejecuta y te saludo, el resto está todo comentado')
# print('Este print no se ejecuta porque está comentado')
```

4. Ejecutar el script completo desde la terminal.

```
python 01_comentarios.py
```

Laboratorio 2. Tipos de datos básicos

1. Abrir el fichero `02_tipos_de_datos_basicos.py` y localizar los ejemplos de enteros.

Python no exige declarar el tipo de una variable. El tipo queda asociado al objeto asignado. Además, el lenguaje diferencia mayúsculas y minúsculas en los nombres de variables.

```
stock_portatiles = 15
# Stock_portatiles = 20 Python es case sensitive; esta sería una variable distinta
stock_monitores = 7
total_equipos = stock_portatiles + stock_monitores
print("Tipo de total_equipos:", type(total_equipos))
print(total_equipos)
```

2. Revisar la diferencia entre concatenar texto y mostrar valores separados por comas.

```
print("Total productos: " + str(total_equipos))
print("Total productos:", total_equipos)
```

La concatenación con `+` exige que todos los elementos sean cadenas. En cambio, `print()` puede recibir varios valores separados por comas y mostrarlos sin convertir manualmente.

3. Revisar los ejemplos de `float`, `bool` y `str`.

```
precio_teclado = 25.99
iva_aplicado = 1.21
precio_final = precio_teclado * iva_aplicado
print("Tipo de precio_final:", type(precio_final))
print("Coste total:", precio_final)

envio_gratis = False
print("¿Tiene envío gratuito?", envio_gratis)
print("Tipo de envio_gratis:", type(envio_gratis))

marca = "Logitech"
modelo = 'G-Pro'
print(marca, modelo)
print(marca + " " + modelo)
```

4. Ejecutar el script y comprobar la salida de `type()`.

```
python 02_tipos_de_datos_basicos.py
```

Nota: si se ejecuta el script completo, aparecerán entradas con `input()` en la parte de conversiones. Introducir valores numéricos válidos para continuar.

Laboratorio 3. Conversiones de tipos

1. Abrir el fichero `03_conversiones_de_tipos.py` y revisar la conversión de texto a entero.

`input()` siempre devuelve `str`. Para operar aritméticamente con el dato leído hay que convertirlo al tipo correcto.

```
valor_leido = input("Introduce la cantidad de productos (numero entero): ")
print("Tipo antes de la conversión:", type(valor_leido))
cantidad = int(valor_leido)
print("Tipo tras conversión:", type(cantidad))

precio_unidad = int(input("Introduce el precio (numero entero): "))
total_compra = cantidad * precio_unidad
```

```
print("Coste total entero:", total_compra)
```

2. Revisar la conversión de texto a número decimal y el uso de `round()`.

```
distancia = float(input("Introduce la distancia en km: "))
resultado_con_math_pow = math.pi * math.pow(distancia, 2)
resultado_con_operador = math.pi * distancia ** 2
print("Resultado con math.pow:", resultado_con_math_pow)
print("Resultado con operador **:", resultado_con_operador)
print("Resultado redondeado:", round(resultado_con_operador, 3))
```

3. Comprobar la conversión a booleanos y cadenas.

```
activado = 0
print("¿Está el sistema activo?:", bool(activado))
activado = 1
print("¿Está el sistema activo?:", bool(activado))

estado_texto = "False"
print("Resultado booleano de la cadena 'False':", bool(estado_texto))

puntuacion = 95
mensaje_final = "Tu nota es: " + str(puntuacion)
print(mensaje_final)
```

4. Ejecutar el script completo y probar entradas correctas e incorrectas.

```
python 03_conversiones_de_tipos.py
```

Laboratorio 4. Manejo de cadenas

1. Abrir el fichero `04_manejo_de_cadenas.py` y revisar la cadena base del ejercicio.

```
mensaje: str = "servidor ACTIVO en la red local"
```

2. Ejecutar transformaciones de formato para normalizar o presentar texto.

```
print(mensaje)
print(mensaje.capitalize())
print(mensaje.title())
print(mensaje.upper())
print(mensaje.lower())
print(mensaje.swapcase())
```

3. Comprobar métodos de validación que devuelven `True` o `False`.

```
print("Cisco123", "isalnum:", 'Cisco123'.isalnum())
print("Admin", "isalpha:", 'Admin'.isalpha())
print("2026", "isdigit:", '2026'.isdigit())
print("ROOT_01", "isupper:", 'ROOT_01'.isupper())
print("admin 80", "islower:", 'admin 80'.islower())
```

4. Revisar funciones generales aplicadas a cadenas.

```
print("Caracteres totales:", len(mensaje))
print("Valor máximo ASCII:", max(mensaje))
print("Valor mínimo ASCII:", min(mensaje))
```

5. Limpiar espacios o caracteres sobrantes en los extremos de una cadena.

```
log_entrada = "      Error de sistema      "
print(log_entrada, end="[FIN]\n")
```

```
print("lstrip:", log_entrada.lstrip(), end="[FIN]\n")
print("rstrip:", log_entrada.rstrip(), end="[FIN]\n")
print("strip:", log_entrada.strip(), end="[FIN]\n")
print("strip de caracter -:", '---LOG-FILE---'.strip('-'))
```

6. Buscar, sustituir, dividir y unir texto.

```
print("replace e -> 3:", mensaje.replace("e", "3"))
print("replace a -> 4 solo la 1ª a:", mensaje.replace("a", "4", 1))
print("find:", mensaje.find('e'))
print("rfind:", mensaje.rfind('e'))
print("count 'a':", mensaje.count('a'))

IP = "192.168.1.50"
IP_split = IP.split(".")
print("IP_split:", IP_split)

partes_ip = ["10", "0", "0", "1"]
print("join:", ".".join(partes_ip))
```

7. Ejecutar el script completo.

```
python 04_manejo_de_cadenas.py
```

Laboratorio 5. Operadores

1. Abrir el fichero 05_operaciones.py y revisar las operaciones aritméticas básicas.

```
unidades = 9
cajas = 4
print("Suma total:", unidades + cajas)
print("Diferencia:", unidades - cajas)
print("Producto:", unidades * cajas)
print("Cociente decimal:", unidades / cajas)
print("Cociente entero:", unidades // cajas)
print("Resto de división entero:", unidades % cajas)
print("Exponencial:", unidades ** cajas)
```

2. Revisar operadores de asignación compuesta.

```
stock = 20
print("Stock:", stock)
stock += 10
print("Stock:", stock)
stock *= 10
print("Stock:", stock)
```

3. Comprobar operadores relacionales y lógicos.

```
print("¿Es mayor?:", stock > unidades)
print("¿Es mayor o igual?:", stock >= 50)
print("¿Es idéntico valor?:", stock == 30)
print("¿Es valor distinto?:", stock != 10)

print("Uso de and:", stock > 0 and unidades == 9)
print("Uso de or:", stock < 0 or unidades > 5)
print("Uso de not:", not unidades == 5)
```

4. Diferenciar identidad, igualdad y pertenencia.

```
carrito_a = ['Monitor', 'Teclado']
carrito_b = ['Monitor', 'Teclado']
print("¿Mismo contenido?:", carrito_a == carrito_b)
```

```
print("¿Misma ubicación de lista?:", carrito_a is carrito_b)

usuarios_pro = ['Admin', 'Editor']
print("¿Admin en lista?:", 'Admin' in usuarios_pro)
print("¿Socio fuera de lista?:", 'Socio' not in usuarios_pro)

puntuacion = 85
print("¿Está en rango 0-100?:", 0 <= puntuacion <= 100)
```

5. Ejecutar el script completo.

```
python 05_operaciones.py
```

Laboratorio 6. Condicionales y match/case

1. Abrir el fichero 06_condicionales.py y revisar la entrada de temperatura.

```
grados = int(input("Ingrese temperatura del nodo (18-27): "))
```

2. Comparar una estructura con condicionales anidados frente a if/elif/else.

```
if grados < 18:
    print("Alerta: Temperatura demasiado baja")
else:
    if grados > 27:
        print("Alerta: Riesgo de sobrecalentamiento")
    else:
        print("Estado del sistema: Óptimo")

if grados < 18:
    print("Alerta: Temperatura demasiado baja")
elif grados > 27:
    print("Alerta: Riesgo de sobrecalentamiento")
else:
    print("Estado del sistema: Óptimo")
```

La segunda estructura suele ser más clara porque evita anidamientos innecesarios. En código profesional conviene priorizar legibilidad frente a condensar demasiado.

3. Revisar una forma abreviada de if en una línea.

```
if grados < 18: print("Alerta: Temperatura demasiado baja")
elif grados > 27: print("Alerta: Riesgo de sobrecalentamiento")
else: print("Estado del sistema: Óptimo")
```

Nota: esta sintaxis existe, pero no siempre mejora la lectura. En scripts de soporte es preferible usar bloques normales cuando hay varias ramas.

4. Revisar match/case para clasificar una prioridad de ticket.

```
codigo = input("Prioridad del ticket [B]aja, [M]edia, [A]lta, [U]rgente: ").lower()
match codigo:
    case 'b':
        print("Prioridad: BAJA. Respuesta en 48 horas.")
    case 'm':
        print("Prioridad: MEDIA. Respuesta en 24 horas.")
    case 'a':
        print("Prioridad: ALTA. Respuesta en 4 horas.")
    case 'u':
        print("Prioridad: CRÍTICA. Respuesta inmediata.")
    case '1' | '2' | '3':
        print("Código numérico detectado. Por favor, use letras.")
    case _:
        print("Código de prioridad no reconocido.")
```

5. Ejecutar el script y probar varios valores de temperatura y prioridad.

```
python 06_condicionales.py
```

Laboratorio 7. Bucles for

1. Abrir el fichero 07_bucles_for.py y revisar el recorrido de una lista.

```
componentes = ['CPU', 'GPU', 'RAM', 'SSD', 'PSU']
print("Lista de componentes:", componentes)

for item in componentes:
    print(item, end=" ")
print()

for item in componentes:
    print(item)
```

2. Usar enumerate() para obtener índice y valor a la vez.

```
for indice, nombre in enumerate(componentes):
    print(f"ID {indice} -> {nombre}")
```

3. Comparar formatted string con concatenación tradicional.

```
for i in range(len(componentes)):
    print(f"Posición {i}: {componentes[i]}")

for i in range(len(componentes)):
    print("Posición " + str(i) + ": " + componentes[i])
```

4. Practicar range() con inicio, fin y paso.

```
for valor in range(10):
    print(valor, end=" ")
print()

for valor in range(5, 16):
    print(valor, end=" ")
print()

for valor in range(10, 21, 2):
    print(valor, end=" ")
print()

for valor in range(5, 0, -1):
```

```
print(valor, end=" ")
```

5. Analizar dos formas de acumular ventas en índices impares.

```
ventas = [120, 450, 300, 210, 580, 600, 150]
total_impares = 0

for i in range(len(ventas)):
    if i % 2 != 0:
        total_impares += ventas[i]
print("Total acumulado (índices impares):", total_impares)

total_impares = 0
for i in range(1, len(ventas), 2):
    total_impares += ventas[i]
print("Total acumulado (optimizado):", total_impares)
```

6. Ejecutar el script completo.

```
python 07_bucles_for.py
```

Laboratorio 8. Bucles while

1. Abrir el fichero 08_bucles_while.py y revisar el contador de intentos.

```
intentos_realizados = 0
max_intentos = 3
```

2. Analizar el bucle de validación de acceso.

```
while intentos_realizados < max_intentos:
    acceso = input("Introduce código de acceso: ")

    if acceso == "admin123":
        print("Acceso concedido al sistema.")
        break

    intentos_realizados += 1
    intentos_restantes = max_intentos - intentos_realizados

    if intentos_restantes > 0:
        print(f"Código erróneo. Le quedan {intentos_restantes} intentos.")
    else:
        print("Acceso bloqueado. Agotó todos los intentos.")
```

break termina el bucle inmediatamente cuando se cumple la condición de acceso. La variable `intentos_realizados` cambia en cada vuelta para evitar un bucle infinito.

3. Revisar el uso de `continue` para saltar una iteración.

```
contador = 0
while contador < 5:
    contador += 1
    if contador == 3:
        continue
    print(f"Procesando paso: {contador}")
```

NOTA: `continue` no termina el bucle: salta el resto del bloque actual y vuelve a evaluar la condición del `while`.

4. Ejecutar el script y probar claves correctas e incorrectas.

```
python 08_bucles_while.py
```

Ejercicios propuestos

1. Crear un script `src/normaliza_usuario.py` que pida un nombre de usuario, elimine espacios al principio y al final, lo convierta a minúsculas y muestre el resultado con una f-string.
2. Crear un script `src/calculo_inventario.py` que pida unidades y precio unitario, calcule el total con IVA del 21% y muestre el importe redondeado a dos decimales.
3. Crear un script `src/clasifica_ticket.py` que pida un código de prioridad y use `match/case` para clasificarlo como baja, media, alta, urgente o no reconocida.
4. Crear un script `src/valida_ip_simple.py` que pida una dirección IP, la divida con `split(".")`, compruebe que tiene cuatro partes y muestre cada octeto en una línea.
5. Crear un script `src/reintento_clave.py` que permita tres intentos para introducir una clave. Si la clave es correcta, debe terminar con `break`; si se agotan los intentos, debe mostrar un mensaje de bloqueo.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno virtual no está activado.	Ejecutar <code>source .venv/bin/activate</code> desde <code>~/Curso_Python/Tema02</code> .
El script no se encuentra	La terminal no está situada en el directorio del tema.	Ejecutar <code>cd ~/Curso_Python/Tema02</code> y comprobar <code>ls -la</code> .
<code>ValueError</code> al convertir con <code>int()</code> o <code>float()</code>	La entrada introducida no tiene el formato esperado.	Repetir la ejecución usando números enteros o decimales válidos según el ejercicio.
<code>IndentationError</code>	Hay espacios o tabulaciones incorrectas en un bloque.	Revisar la alineación del código. Usar 4 espacios por nivel y no mezclar tabs.
<code>NameError</code>	Se ha escrito una variable con otro nombre o diferente capitalización.	Comprobar mayúsculas y minúsculas. Python diferencia <code>stock</code> de <code>Stock</code> .
El notebook usa otro Python	El kernel seleccionado no es el del entorno del tema.	Seleccionar el kernel asociado a <code>.venv/bin/python</code> o verificar <code>sys.executable</code> .